

Freight Rate Prediction

Machine Learning Engineer assessment

Summary

A two-stage additive model in log space predicts `posted_rate` with a mean absolute percentage error of **1.61%** on held-out non-corrupted rows, averaged over four chronological validation folds, against **3.93%** for a specification without temporal features. The improvement is associated with one identified mechanism: rates follow a *quarterly cycle* that is flat for roughly eight weeks and then ramps about 5% into quarter-end. Encoding time as position-within-quarter rather than as a calendar date lets the November–December horizon be projected from the pattern learned in earlier quarters, instead of extrapolated blindly in calendar time.

Two data-quality defects were found and addressed: a sign flip affecting 292 `weight` values, and multiplicative corruption of 1.41% of `posted_rate` labels.

1 Data exploration and quality

The development set holds 48,000 labelled loads spanning 2025-01-01 to 2025-10-31, with no duplicate `load_id` values and no gaps in date coverage. The validation set holds 12,000 loads over 2025-11-01 to 2025-12-31, giving a strictly forward 61-day prediction horizon with zero date overlap.

1.1 Defect 1: sign-flipped weights

292 rows carry negative `weight`. The absolute values of these observations reproduce the distribution of the valid ones almost exactly (median 31 821 versus 31 494; both bounded at 5000 and 47 500), which identifies a sign flip rather than random corruption. The values were repaired with `abs()` rather than dropping the rows, which preserves 292 usable labels.

1.2 Defect 2: multiplicative corruption of the target

`posted_rate` follows a tight power law in distance. Fitting $\log(\text{rate}) \sim \log(\text{distance})$ by iteratively reweighted least squares gives $\log(\text{rate}) = 1.651 + 0.871 \log(\text{distance})$ with a robust residual scale of $\sigma = 0.079$. The residual distribution (Figure 1) is not heavy-tailed — it is *disjoint*. The core lies inside $|r| < 0.25$, and only 2 of 48,000 rows fall in the band $0.25 < |r| < 0.70$.

Converting the flagged residuals to multipliers (Figure 2) identifies the mechanism: 340 rows inflated and 337 deflated, with factors distributed continuously over roughly $\times 2$ to $\times 5.5$ and their reciprocals, and nothing between 0.5 and 2. The flagged rows show no distributional signature in distance, weight, equipment, origin city or month. This is injected noise on the label, not legitimate price variation.

A rate-per-mile cutoff was rejected as a detection rule, because rate per mile is strongly distance-dependent (correlation -0.68 with distance): a fixed threshold would systematically misclassify short hauls. The log-residual rule is distance-invariant by construction.

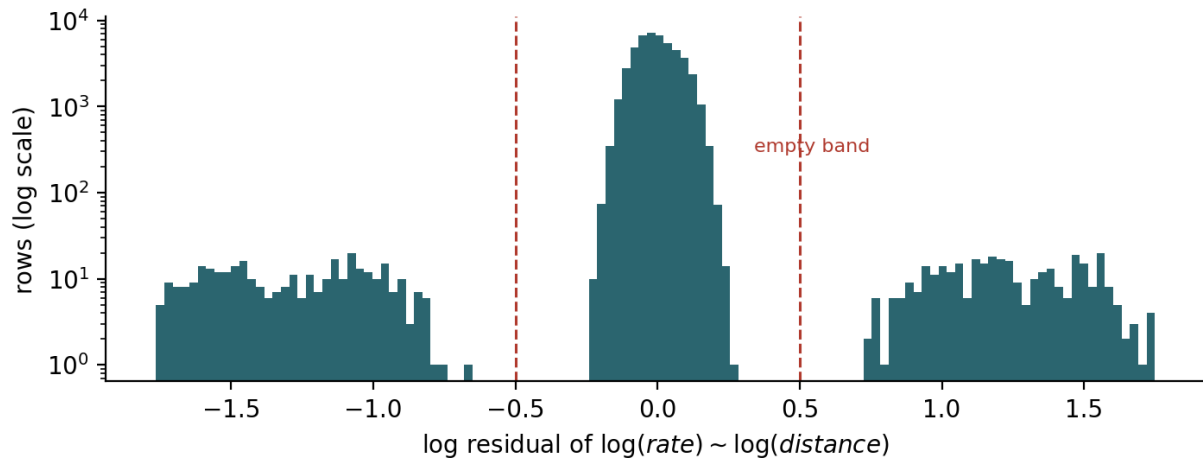


Figure 1: Log-residual distribution (log-scaled counts). The dashed lines mark the ± 0.5 threshold, chosen inside an empty band rather than at an arbitrary percentile.

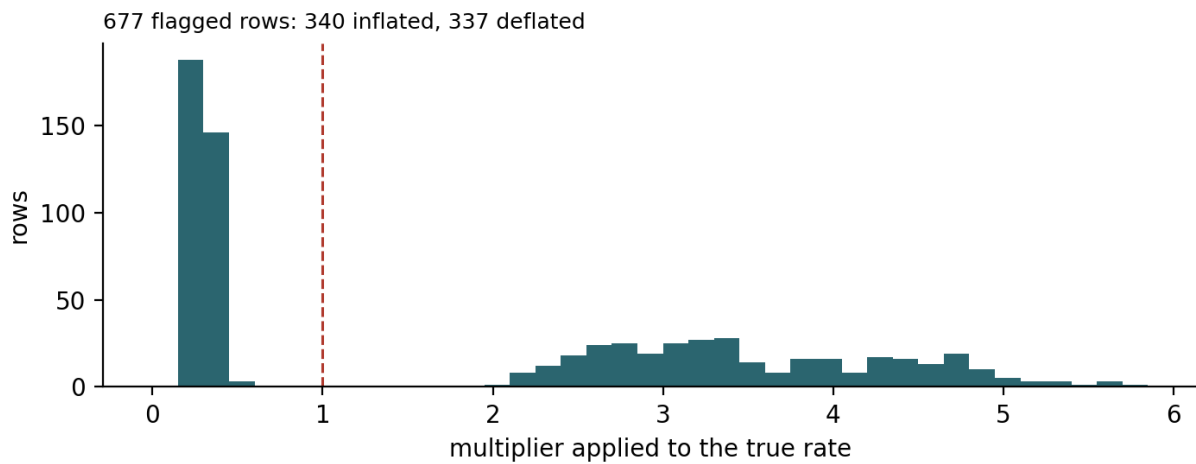


Figure 2: Multiplier implied by each flagged row. The symmetry and the absent region around 1 identify random multiplicative corruption of the target.

1.3 Missing values

300 missing `weight` and 374 missing `market_index` values in development; 165 and 249 in validation. Mean `posted_rate` is statistically indistinguishable between present and missing groups (\$2374 vs \$2351, \$2374 vs \$2392), consistent with missingness completely at random, so simple imputation is adequate. `weight` is filled with the median for its equipment type; `market_index` is filled from that day's mean, for the reason given next. Binary missingness flags were retained.

1.4 `market_index` is a daily signal

Within-day standard deviation of `market_index` is 0.025, against an annual range of 0.68 to 1.47. Each row therefore carries a noisy reading of one underlying daily value, and the daily mean is both a better imputation source and a less noisy model input than the raw column. In December the daily index oscillates on a clear seven-day cycle.

1.5 Features that do not earn a place

`quote_signal` improves in-sample residual scale from 0.03991 to 0.03978 — 0.3% on 47,000 rows — and made held-out performance *worse* on three of four validation folds. It was dropped. This also removes the model's last dependency on a column absent from the December scenario file.

Eight cities (Chicago, Charlotte, San Diego, Laredo, Knoxville, Allentown, Norfolk, Jackson) appear only in validation, covering 1,447 rows or 12% of the scored set. Any target encoding or one-hot encoding on city name fails on these rows. Geography is therefore admitted only through coordinates, which generalise to unseen cities; a linear function of the four coordinate columns explains 27.7% of the residual variance remaining after the core elasticities.

2 Temporal structure: the central finding

After fitting the elasticities, the residual retains strong temporal structure. Averaged by week within quarter (Figure 3), Q1, Q2 and Q3 trace nearly the same path: flat for eight weeks, then a ramp of roughly +0.05 in log space over the final four. Quarter levels step up almost linearly: -0.033 , -0.015 , $+0.003$, $+0.024$, i.e. steps of $+0.018$, $+0.019$, $+0.021$.

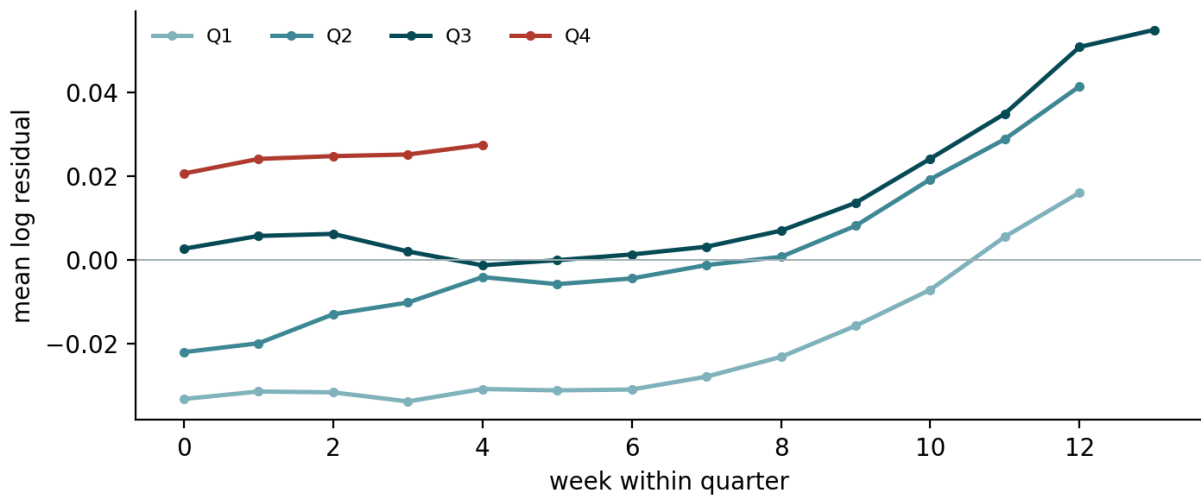


Figure 3: Mean log residual by week within quarter. The within-quarter shape repeats; the level steps up each quarter. Q4 (red) is observed only through 31 October — its first five week-bins — which is what anchors its level.

This determines the feature design. Table 1 shows why.

Table 1: Day-of-quarter coverage. Every value required at prediction time is observed in training — in a different quarter.

		day-of-quarter	provides
Training	Q1–Q3	0–91	the full within-quarter ramp, three times the Q4 level
Training	Q4 (October)	0–30	
Required	Q4 (Nov–Dec)	31–91	—

Encoded as a calendar date or as month dummies, November and December lie outside the training range entirely and any model flattens to the October level, under-predicting by around 5%. Encoded as `(day_of_quarter, trend)`, the required region is reachable: the within-quarter shape is *interpolated*, since day-of-quarter 61–91 is densely observed in Q1–Q3, and only the level drift is extrapolated — along a single monotone dimension, anchored by October’s observed position.

A day-of-month effect was also visible in raw form, but Figure 3 shows no monthly reset — that pattern was the third month of each quarter showing through. Since December is simultaneously month-end and quarter-end, the two were tested separately; `day_of_quarter` carries the signal and `day_of_month` was excluded.

3 Validation design

3.1 Split

The scored task is a forward 61-day horizon from a training set ending 31 October. A random split would allow same-period rows to predict each other and produce an optimistic and uninformative estimate. All validation is therefore strictly chronological, over four folds:

Table 2: Validation folds. Fold D is a structural analogue of the scored task.

Fold	Train	Predict	Tests
A	Jan 1 – Apr 30	May 1 – Jun 30	a general 61-day horizon
B	Jan 1 – Jun 30	Jul 1 – Aug 31	crossing a quarter boundary
C	Jan 1 – Aug 31	Sep 1 – Oct 31	primary mirror of the scored task
D	Jan 1 – Jul 31	Sep 1 – Sep 30	the December analogue

Fold D deserves explanation. December is Q4 days 61–91, with its quarter level anchored by October (days 0–30) and November skipped in between. Fold D reproduces that geometry exactly inside labelled data: train through July (so Q3 days 0–30 are observed), skip August, predict September (Q3 days 62–91). It is the fold that most closely resembles what is being submitted, and — as a check that the design is not self-flattering — it is the fold on which the naive baseline performs worst.

3.2 Two rules

Corrupted rows are removed from training only, never from a held-out fold. The scored set contains all 12,000 rows, corruption included. Cleaning the evaluation side would produce a validation figure that could not be reproduced by the grading scorer.

Nothing derived from the target crosses a fold boundary. The outlier rule is fitted inside the training fold. Refitting it on a training fold alone gives $1.647 + 0.871 \log(d)$ against $1.651 + 0.871 \log(d)$ on the

full set, flags the same 677 rows, and disagrees on none — so the rule is demonstrably invariant to the boundary rather than merely argued to be.

The daily `market_index` table is built from all available feature rows, including validation. This is not leakage: `market_index` is an input column supplied for every row requiring a prediction, no target values enter the aggregation, and the operation is identical at internal-validation and final-prediction time. The stricter alternative — deriving it from training months only — would require forecasting the index for November and December and would discard real data supplied with the task.

3.3 Metric selection

Root mean squared error was rejected for model selection. Simulating a *perfect* model against corrupted targets gives $\text{RMSE} \approx \$657$ and $\text{MAPE} \approx 2.3\%$ purely from the 1.41% unpredictable rows. Worse, across simulated models of log-residual scale 0.00, 0.02, 0.03 and 0.04, RMSE came out \$657, \$761, \$631, \$690 — non-monotonic in model quality. The observed fold results confirm it: RMSE varied only 4% (\$610–\$634) across architectures whose clean MAE differed by a factor of 2.4.

Selection therefore used **MAE** and **MAPE**, each reported twice: over all rows (comparable to the grading scorer) and over non-corrupted rows (the model’s real accuracy).

4 Model

4.1 Functional form

Rate is built multiplicatively, so the model is additive in logs:

$$\log(\text{rate}) = 0.759 + 0.871 \log(\text{dist}) + 0.140 \log(\text{mkt}) + 0.081 \log(\text{wt}) + 0.122 [\text{Reefer}] + 0.080 [\text{Flatbed}] \quad (1)$$

$R^2 = 0.9962$, residual scale 0.040. Every coefficient is an elasticity: rate per mile falls with length of haul ($0.871 < 1$), Reefer carries a 13% premium over Dry Van and Flatbed 8%, and a 10% rise in the market index lifts rate by 1.4%.

4.2 Architecture: additive hybrid

The submitted model is a two-stage additive fit in log space:

$$\log(\widehat{\text{rate}}) = \underbrace{f_{\text{lin}}(x)}_{\text{OLS}} + \underbrace{g_{\text{gbm}}(x)}_{\text{residual of } f_{\text{lin}}}, \quad \widehat{\text{rate}} = \exp(f_{\text{lin}}(x) + g_{\text{gbm}}(x)). \quad (2)$$

The linear stage carries everything that must extrapolate: the elasticities, linear geography, a linear trend term, and a cubic in day-of-quarter. A gradient-boosted tree ensemble then fits the *residual* of that model, capturing curvature in distance and weight, regional pricing, and equipment interactions. Because the sum is taken in log space, the tree’s output acts as a multiplier on the dollar scale; its corrections have standard deviation 0.019 and range -8.7% to $+9.6\%$.

The trend term is deliberately withheld from the tree stage. A linear term extends beyond the training window by construction; a tree cannot, since every future date falls into the leaf defined by its last observed split. Placing the trend in the linear stage lets December inherit it correctly, and excluding it from the tree prevents a spurious flat extrapolation. Across the evaluated folds the tree’s corrections remained small (standard deviation 0.019, range -8.7% to $+9.6\%$), so the linear component dominated in the regions requiring extrapolation — an empirical observation rather than a guaranteed property of the ensemble.

4.3 Alternatives tested and rejected

Table 3: Clean-row MAPE by fold. Selection metric: mean across folds, with fold D weighted most heavily as the structural analogue of the scored task.

Model	A	B	C	D	mean
No temporal features (baseline)	2.76%	2.49%	4.64%	5.84%	3.93%
Linear + temporal features	2.12%	2.38%	2.17%	2.21%	2.22%
Gradient boosting alone	2.32%	2.30%	2.08%	1.73%	2.11%
Additive hybrid (submitted)	1.43%	1.82%	1.56%	1.61%	1.61%

Beyond the architecture comparison, four refinements were tested and none improved the mean by more than 0.01 percentage points: a stepwise quarter-level term in place of the daily trend; recalibration of the level on the most recent 30 training days; `quote_signal` as an additional regressor; and four ramp bases (cubic, quartic, single hinge, double hinge). The simplest specification was retained.

One prior expectation was tested and disproved. Gradient boosting alone was expected to fail on fold D, because the cell *Q4 and day-of-quarter > 30* contains no training rows and a tree cannot extend an additive trend across an empty cell. Fold D was in fact its strongest fold. The mechanism is real but small here, since quarter levels are only 0.019 apart. The hybrid was selected on measured accuracy, not on that argument.

4.4 Residual bias

The hybrid retains a level bias of -0.007 on fold D. Broken out by week of quarter it is flat (-0.006 to -0.010), so it is not a misfit of the ramp shape — the shape is correct, which is what governs the December curve. Its sign varies across folds (-0.012 , $+0.005$, -0.007), so it is not a correctable bias but residual level uncertainty of roughly $\pm 1\%$ at a 61-day horizon. Correcting it would mean fitting to a single fold.

5 December scenario

The December file fixes pickup, delivery, distance, equipment and weight, so date supplies the entire shape of the curve through two channels: the daily market index (a seven-day cycle, $\pm 1.6\%$ on rate) and position within quarter (Dec 1 is day 61, Dec 31 is day 91 — the steep section of the ramp). Day of week was measured at under 0.5% and is immaterial. A model without temporal features would produce a flat line, which the data show to be wrong.

Coordinates for Lexington and Fort Wayne are recovered from the city table; the daily market index for each December date comes from the validation feature rows. The prediction rises from \$828.05 on 1 December to \$879.93 on 31 December, $+6.3\%$ across the month, with four weekly oscillations (Figure 4).

As a consistency check, the same shape was projected independently from the Step-3 linear decomposition before any tree was fitted, giving \$828 to \$896 ($+8.2\%$). The two agree on level and direction, and differ in slope by the amount the tree corrects.

Predicted rate per mile has a median of 2.176 against 2.145 for clean training rows, a $+1.4\%$ difference, with near-identical distance and equipment mixes between the two sets. That gap is the time trend, of the expected magnitude for a horizon a quarter further along.

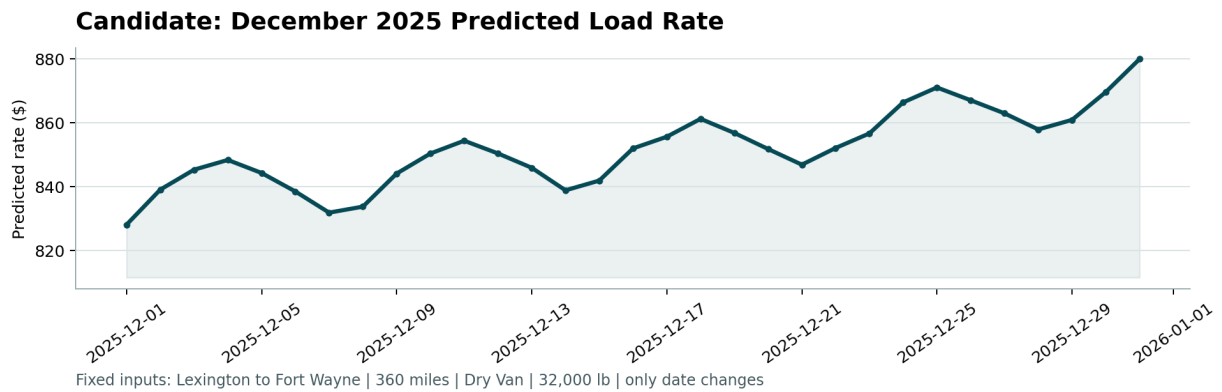


Figure 4: Output of the provided `score.py` for the fixed December scenario.

6 Limitations and next steps

- Level uncertainty of about $\pm 1\%$ at a 61-day horizon is irreducible from a single year of data; a second year would separate the quarterly cycle from an annual one, which cannot be distinguished here.
- The corrupted 1.41% of validation labels are unpredictable by construction and will inflate any reported error metric. Reported all-row MAPE near 5% against clean-row 1.6% reflects this floor rather than model error.
- If the grading metric is RMSE in dollars, a small upward adjustment ($\approx +1.4\%$, the expected value of the corruption multiplier) would reduce it. This was not applied: it optimises against injected noise, degrades MAE, and would misrepresent the model's estimate of the true rate.
- Quantile or conformal intervals would be a natural extension, since brokerage pricing decisions depend on the spread as much as the point estimate.

Reproduction

```
python -m pip install -r requirements.txt
python src/pipeline.py --data-dir data --out-dir .
python score.py --predictions validation_predictions.csv \
    --december-predictions data/december_chart_inputs.csv
```